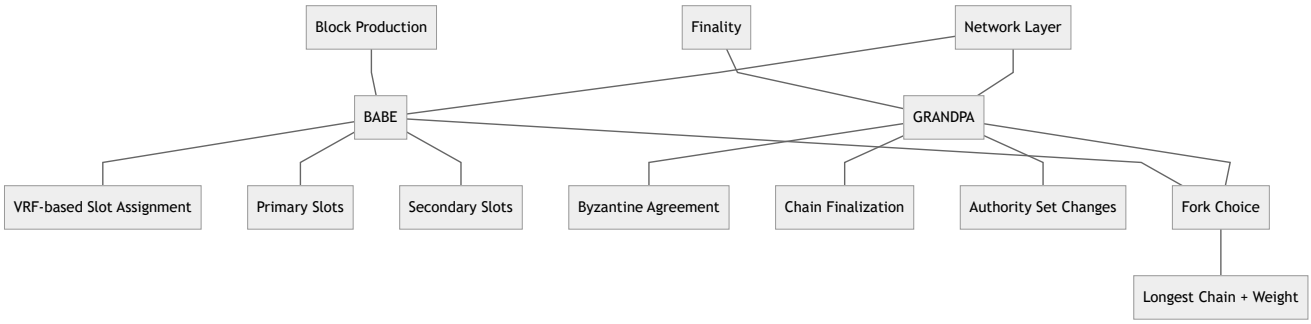
2.4 합의 메커니즘 (BABE, GRANDPA) 구현 분석

개요

Substrate는 블록 생성과 최종성을 분리한 하이브리드 합의 시스템을 사용합니다. BABE(Blind Assignment for Blockchain Extension)는 블록 생성을 담당하고, GRANDPA(GHOST-based Recursive ANcestor Deriving Prefix Agreement)는 최종성을 제공합니다. 이 문서에서는 두 합의 메커니즘의 동작 원리, 구현 세부사항, 그리고 상호작용 방식을 상세히 분석합니다.

하이브리드 합의 아키텍처

합의 시스템 구조



분리된 합의의 장점

1. **확장성**: 블록 생성과 최종성이 독립적으로 작동
2. **유연성**: 각 메커니즘을 독립적으로 최적화 가능

3. **안전성**: 최종성은 더 강한 보안 보장 제공

4. **성능**: 빠른 블록 생성과 안전한 최종성의 균형

BABE (Blind Assignment for Blockchain Extension)

BABE 개념 및 원리

BABE는 VRF(Verifiable Random Function) 기반의 슬롯 할당 메커니즘을 사용하여 블록을 생성합니다.

핵심 개념

슬롯 (Slot)

- 고정된 시간 간격 (일반적으로 6초)
- 각 슬롯에서 하나 이상의 검증자가 블록을 생성할 수 있음

에포크 (Epoch)

- 연속된 슬롯들의 집합 (일반적으로 2400 슬롯 = 4시간)
- 동일한 검증자 세트와 랜덤성을 사용

VRF (Verifiable Random Function)

- 검증 가능한 랜덤 함수
- 슬롯 할당의 공정성과 검증 가능성 보장

BABE 알고리즘 동작 원리

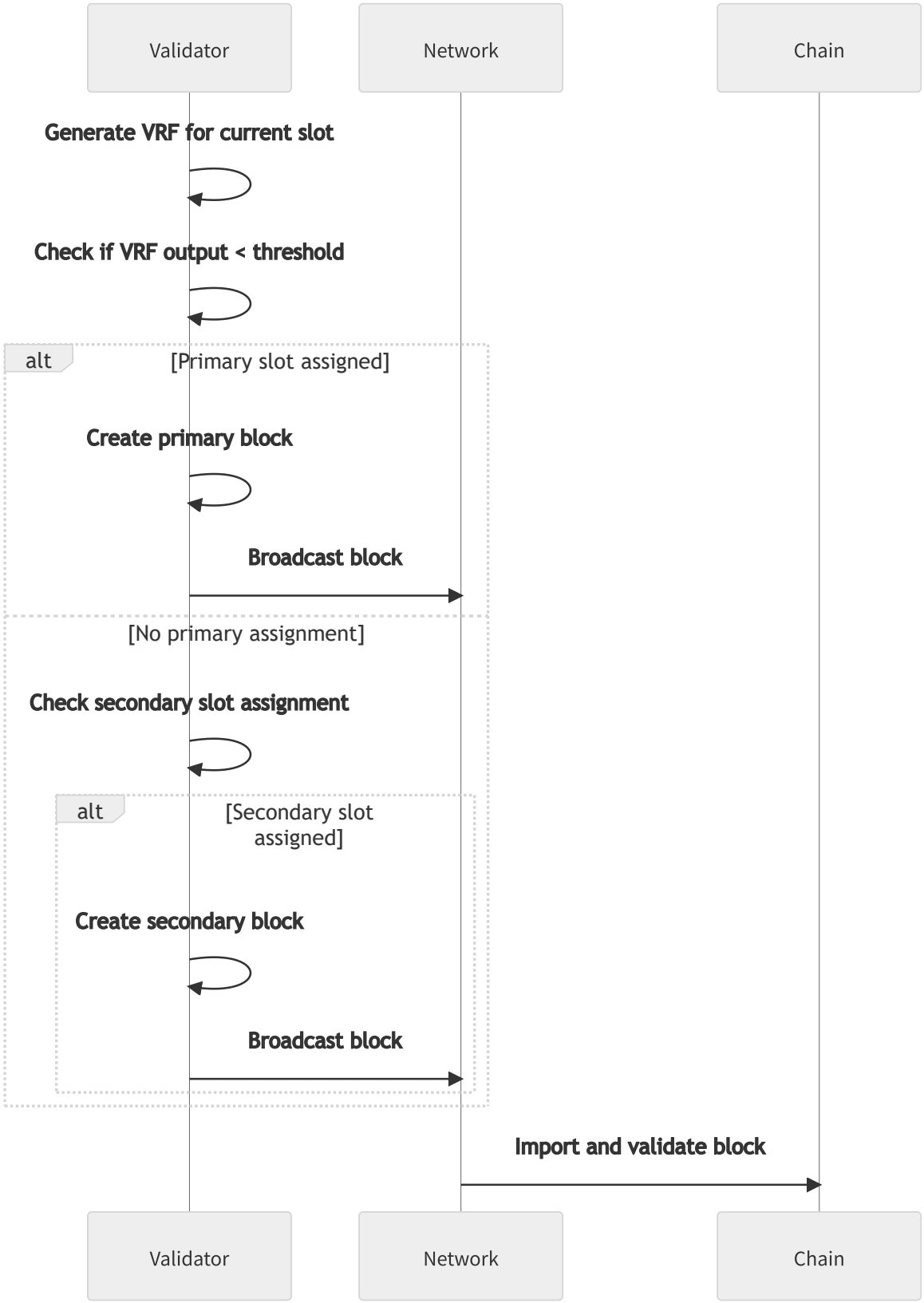
1. 슬롯 할당 과정

```
// VRF 입력 생성
pub fn make_vrf_transcript(randomness: &Randomness, slot: Slot, epoch: u64) ->
```

```
VrfInput::new(
&BABE_ENGINE_ID,
&[
(b"slot number", &slot.to_le_bytes()),
(b"current epoch", &epoch.to_le_bytes()),
(b"chain randomness", randomness),
],
)
}

// 슬롯 할당 검증
fn  check_primary_threshold(
vrf_output: &VrfPreOutput,
authorities_len: usize,
authority_weight: u64,
total_weight: u64,
c: (u64, u64), // 확률 매개변수
) -> bool {
let  threshold = calculate_threshold(c, authorities_len, authority_weight, total
vrf_output.make_bytes::<[u8; 32]>(BABE_VRF_PREFIX) < threshold
}
```

2. 블록 생성 과정



3. Primary vs Secondary 슬롯

Primary 슬롯

- VRF 기반 확률적 할당
- 높은 가중치 (포크 선택에서 우선순위)
- 체인 보안의 핵심

```
// Primary 슬롯 검증
impl<T: Config> Pallet<T> {
fn check_primary_slot(
slot: Slot,
authority_index: u32,
authority_id: &AuthorityId,
randomness: &Randomness,
) -> Option<VrfPreOutput> {
let transcript = make_vrf_transcript(randomness, slot, Self::epoch_index());
// VRF 서명 생성 및 검증
if let Ok((vrf_output, vrf_proof)) = authority_id.vrf_sign(&transcript) {
if Self::check_primary_threshold(&vrf_output, /* ... */) {
return Some(vrf_output);
}
}
None
}
}
```

Secondary 슬롯

- 결정론적 할당
- 낮은 가중치 (백업 메커니즘)
- 빈 슬롯 방지

```
// Secondary 슬롯 할당
fn secondary_slot_author(
slot: Slot,
authorities: &[AuthorityId],
randomness: &Randomness,
) -> Option<&AuthorityId> {
let rand_bytes = blake2_256(&(randomness, slot).encode());
let rand = u32::from_le_bytes([rand_bytes[0], rand_bytes[1], rand_bytes[2], ran
let authority_index = (rand as usize) % authorities.len();
authorities.get(authority_index)
}
```

BABE 구현 구조

클라이언트 사이드 구현

```
// BABE 워커 구조
pub struct BabeWorker<B, E, I, SO, SC, CAW, BS> {
client: Arc<I>,
```

```
block_import: I,

env: E,

sync_oracle: SO,

justification_sync_link: SC,

force_authoring: bool,

backoff_authoring_blocks: Option<BS>,

keystore: KeystorePtr,

epoch_changes: SharedEpochChanges<B, Epoch>,

config: Config,

}



impl<B, E, I, SO, SC, CAW, BS> BabeWorker<B, E, I, SO, SC, CAW, BS>

where

B: BlockT,

I: BlockImport<B> + Send + Sync + 'static,

E: Environment<B> + Send + Sync + 'static,

SO: SyncOracle + Send + Sync + Clone + 'static,

{

// 슬롯 워커 실행

async  fn  start_slot_worker(self) -> Result<(), ConsensusError> {

let  slot_duration = SlotDuration::from_millis(self.config.slot_duration);

let  slot_worker = SlotWorker {

client: self.client.clone(),
```

```
block_import: self.block_import,
env: self.env,
sync_oracle: self.sync_oracle,
force_authoring: self.force_authoring,
backoff_authoring_blocks: self.backoff_authoring_blocks,
keystore: self.keystore,
epoch_changes: self.epoch_changes,
config: self.config,
};

slot_worker.start(slot_duration).await
}
}
```

런타임 사이드 구현

```
// BABE 팔렛 구성
#[pallet::config]
pub  trait Config: pallet_timestamp::Config {
/// 에포크 지속 시간 (슬롯 단위)
#[pallet::constant]
type EpochDuration: Get<u64>;

/// 예상 블록 시간
```

```
#[pallet::constant]
type ExpectedBlockTime: Get<Self::Moment>;

/// 에포크 변경 트리거
type EpochChangeTrigger: EpochChangeTrigger;

/// 비활성화된 검증자 확인
type DisabledValidators: DisabledValidators;

/// 가중치 정보
type WeightInfo: WeightInfo;

/// 키 소유권 증명
type KeyOwnerProof: Parameter + GetSessionNumber + GetValidatorCount;

/// 등가 증명 보고 시스템
type EquivocationReportSystem: OffenceReportSystem<
Option<Self::AccountId>,
(EquivocationProof<Self::Hash, BlockNumberFor<Self>>, Self::KeyOwnerProof),
>;
}

// 에포크 변경 처리
```

```
impl<T: Config> Pallet<T> {
pub fn do_initialize(now: BlockNumberFor<T>) {
// 현재 에포크 정보 업데이트
Self::deposit_consensus_digest(ConsensusLog::NextEpochData(NextEpochDescriptor {
authorities: Self::next_authorities().to_vec(),
randomness: Self::next_randomness(),
}));

// 에포크 변경 트리거 실행
T::EpochChangeTrigger::trigger::<T>(now);
}

fn enact_epoch_change(
authorities: WeakBoundedVec<AuthorityId, T::MaxAuthorities>,
next_authorities: WeakBoundedVec<AuthorityId, T::MaxAuthorities>,
session_index: Option<SessionIndex>,
) {
// 새 에포크로 전환
Authorities::<T>::put(&next_authorities);
// 랜덤성 업데이트
let next_randomness = Self::randomness_change_epoch(authorities.len() as u32);
NextRandomness::<T>::put(&next_randomness);
// 에포크 인덱스 증가
```

```
let  epoch_index = EpochIndex::<T>::get().saturating_add(One::one());

EpochIndex::<T>::put(&epoch_index);

}

}
```

GRANDPA (GHOST-based Recursive ANcestor Deriving Prefix Agreement)

GRANDPA 개념 및 원리

GRANDPA는 비잔틴 장애 허용 최종성 가젯으로, 체인의 특정 블록에 대해 최종성을 제공합니다.

핵심 개념

최종성 (Finality)

- 블록이 되돌릴 수 없음을 보장
- 2/3 이상의 검증자 동의 필요

라운드 (Round)

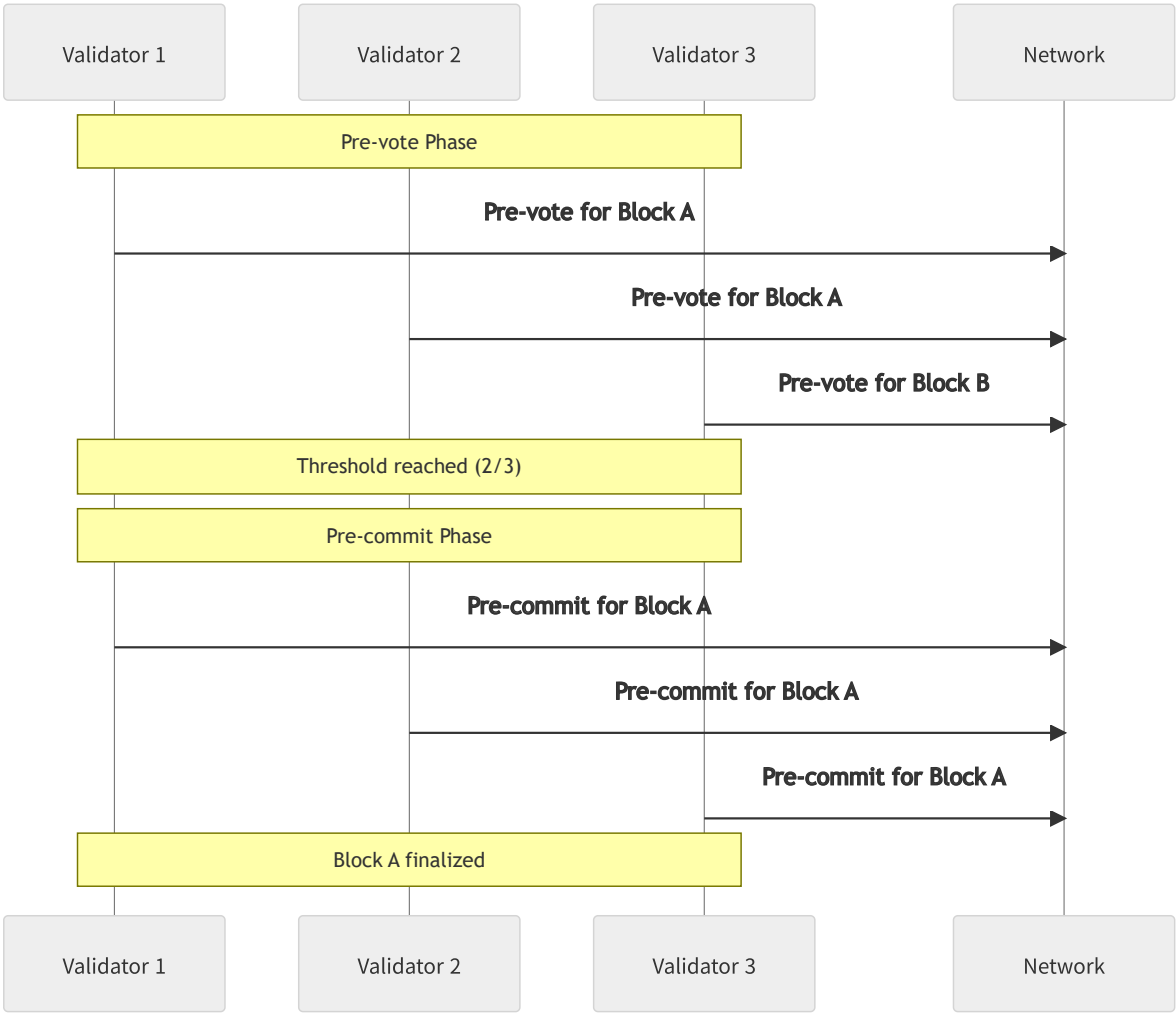
- 최종성 투표의 단위
- 각 라운드에서 하나의 블록을 최종화

투표 (Vote)

- Pre-vote: 후보 블록 제안
- Pre-commit: 최종 블록 확정

GRANDPA 알고리즘 동작 원리

1. 투표 과정



2. GHOST 규칙

GRANDPA는 GHOST(Greedy Heaviest Observed SubTree) 규칙을 사용하여 최적의 체인을 선택합니다:

```
// GHOST 규칙 구현

fn ghost_find_common_ancestor<B: BlockT>(

base: &B::Hash,

votes: &[SignedMessage<B>],
```

```
ancestry: &dyn Ancestry<B>,
) -> Result<B::Hash, Error> {
let mut current = *base;
loop {
let children = ancestry.children(current)?;
if children.is_empty() {
return Ok(current);
}
// 가장 많은 투표를 받은 자식 선택
let mut best_child = None;
let mut best_weight = 0;
for child in children {
let weight = count_votes_for_chain(&child, votes, ancestry)?;
if weight > best_weight {
best_weight = weight;
best_child = Some(child);
}
}
match best_child {
Some(child) if best_weight >= threshold => current = child,
_ => return Ok(current),
}
}
```

```
}
```

3. 최종성 증명

```
// 최종성 증명 구조
#[derive(Clone, Debug, PartialEq, Eq, Encode, Decode)]
pub struct FinalityProof<H> {
/// 라운드 번호
pub round: u64,
/// Pre-commit 투표들
pub commits: Vec<SignedPrecommit<H>>,
/// 투표 증명
pub votes_ancestries: Vec<H>,
}

// 최종성 검증
impl<H: Hash> FinalityProof<H> {
pub fn verify(&self, set_id: u64, voters: &VoterSet<AuthorityId>) -> Result<H,
// 투표 서명 검증
for commit in &self.commits {
if !voters.contains(&commit.id) {
return Err(Error::InvalidVoter);
}
```

```
if !commit.signature.verify(&commit.precommit, &commit.id) {
return Err(Error::InvalidSignature);
}
}
// 2/3 임계값 확인
let total_weight: u64 = voters.total_weight();
let commit_weight: u64 = self.commits.iter()
.map(|c| voters.weight(&c.id).unwrap_or(0))
.sum();
if commit_weight * 3 < total_weight * 2 {
return Err(Error::InsufficientWeight);
}
// GHOST 규칙으로 최종 블록 결정
let target = self.find_ghost_target()?;
Ok(target)
}
}
```

GRANDPA 구현 구조

클라이언트 사이드 구현

```
// GRANDPA 워커 구조
pub struct GrandpaWorker<B, E, N, RA, SC> {
```

```
client: Arc<dyn Client<B>>,
config: Config,
network: N,
select_chain: SC,
voting_rule: Box<dyn VotingRule<B>>,
env: Arc<E>,
voter_set_state: SharedVoterSetState<B>,
}

impl<B, E, N, RA, SC> GrandpaWorker<B, E, N, RA, SC>
where
B: BlockT,
E: Environment<B>,
N: NetworkT<B>,
SC: SelectChain<B>,
{
// 투표 라운드 실행
async fn run_voter(
&mut self,
voter_set: VoterSet<AuthorityId>,
round_number: u64,
) -> Result<(), Error> {
let round_data = RoundData {
```

```
voter_set,
round_number,
prevote_ghost: None,
precommit_ghost: None,
};
// Pre-vote 단계
let prevote_timer = Delay::new(Duration::from_millis(self.config.prevote_timeou
let prevote = self.construct_prevote(&round_data).await?;
self.network.send_prevote(prevote.clone()).await?;
// Pre-vote 수집 및 GHOST 계산
let prevote_ghost = self.collect_prevotes(&round_data, prevote_timer).await?;
// Pre-commit 단계
let precommit_timer = Delay::new(Duration::from_millis(self.config.precommit_ti
let precommit = self.construct_precommit(&round_data, &prevote_ghost).await?;
self.network.send_precommit(precommit.clone()).await?;
// Pre-commit 수집 및 최종성 결정
let commit = self.collect_precommits(&round_data, precommit_timer).await?;
if let Some(finalized_block) = commit {
self.finalize_block(finalized_block).await?;
}
Ok(())
}
```

```
}
```

런타임 사이드 구현

```
// GRANDPA 팔렛 구성
#[pallet::config]
pub trait Config: frame_system::Config {
/// 이벤트 타입
type RuntimeEvent: From<Event> + Into<<Self as frame_system::Config>::RuntimeE


/// 가중치 정보
type WeightInfo: WeightInfo;


/// 최대 권한자 수
#[pallet::constant]
type MaxAuthorities: Get<u32>;


/// 최대 지명자 수
#[pallet::constant]
type MaxNominators: Get<u32>;


/// 키 소유권 증명
type KeyOwnerProof: Parameter + GetSessionNumber + GetValidatorCount;
```

```
/// 등가 증명 보고 시스템
type EquivocationReportSystem: OffenceReportSystem<
Option<Self::AccountId>,
(EquivocationProof<Self::Hash, BlockNumberFor<Self>>, Self::KeyOwnerProof),
>;
}


// 권한자 세트 변경 처리
impl<T: Config> Pallet<T> {
/// 권한자 세트 변경 예약
pub fn schedule_change(
next_authorities: AuthorityList,
in_blocks: BlockNumberFor<T>,
forced: Option<BlockNumberFor<T>>,
) -> DispatchResult {
ensure!(!next_authorities.is_empty(), Error::<T>::EmptyAuthorityList);
let scheduled_at = <frame_system::Pallet<T>>::block_number();
let pending_change = StoredPendingChange {
scheduled_at,
delay: in_blocks,
next_authorities: next_authorities.try_into()
.map_err(|_| Error::<T>::TooManyAuthorities)?,
forced,
```

```
};

PendingChange::<T>::put(&pending_change);

Self::deposit_event(Event::NewAuthorities { authority_set: next_authorities });

Ok(())

}

/// 최종성 로그 생성

fn  deposit_log(log: ConsensusLog<BlockNumberFor<T>>) {

let  log = DigestItem::Consensus(GRANDPA_ENGINE_ID, log.encode());

<frame_system::Pallet<T>>::deposit_log(log);

}

}
```

BABE와 GRANDPA의 상호작용

1. 에포크 동기화

```
// 세션 변경 시 두 시스템 모두 업데이트

impl<T: Config> OneSessionHandler<T::AccountId> for Pallet<T> {

type Key = AuthorityId;



fn  on_new_session<'a, I: 'a>(

changed: bool,

validators: I,
```

```
queued_validators: I,
) where
I: Iterator<Item = (&'a T::AccountId, Self::Key)>,
{
if changed {
let next_authorities = validators.map(|(_, k)| k).collect::<Vec<_>>();
// BABE 권한자 업데이트
let next_authorities_babe = next_authorities.clone();
pallet_babe::Pallet::<T>::enact_epoch_change(
Authorities::<T>::get(),
next_authorities_babe.try_into().unwrap(),
Some(Self::session_index()),
);
// GRANDPA 권한자 업데이트
let next_authorities_grandpa = next_authorities.into_iter()
.map(|id| (id, 1)) // 동일한 가중치
.collect();
Self::schedule_change(next_authorities_grandpa, Zero::zero(), None).ok();
}
}
}
```

2. 포크 선택 규칙

```
// BABE 가중치 + GRANDPA 최종성을 고려한 포크 선택
pub struct BabeGrandpaForkChoice<B, Backend> {
backend: Arc<Backend>,
_phantom: PhantomData<B>,
}

impl<B, Backend> SelectChain<B> for BabeGrandpaForkChoice<B, Backend>
where
B: BlockT,
Backend: AuxStore + HeaderBackend<B> + 'static,
{
fn select_chain(&self) -> Result<B::Hash, ConsensusError> {
// 1. GRANDPA 최종화된 블록 확인
let finalized_hash = self.backend.last_finalized()?;
let finalized_number = self.backend.number(finalized_hash)?
.ok_or(ConsensusError::InvalidBlock)?;
// 2. 최종화된 블록 이후의 최적 체인 선택
let leaves = self.backend.leaves()?;
let mut best_chain = finalized_hash;
let mut best_weight = 0;
for leaf in leaves {
if self.backend.number(leaf)? <= Some(finalized_number) {
```

```
continue;
}
// BABE 가중치 계산 (Primary 블록 = 1, Secondary 블록 = 0)
let weight = self.calculate_babe_weight(finalized_hash, leaf)?;
if weight > best_weight {
best_weight = weight;
best_chain = leaf;
}
}
Ok(best_chain)
}
}
```

검증자 선택 및 슬래싱 메커니즘

1. 검증자 선택

```
// 스테이킹 팔렛과 연동된 검증자 선택
impl<T: Config> pallet_session::SessionManager<T::AccountId> for Pallet<T> {
fn new_session(new_index: SessionIndex) -> Option<Vec<T::AccountId>> {
// 스테이킹에서 검증자 목록 가져오기
let validators = pallet_staking::Pallet::<T>::new_session(new_index)?;
// BABE 및 GRANDPA 키 확인
```

```
let validators_with_keys = validators.into_iter()
.filter_map(|v| {
let babe_key = T::BabeAuthorityId::from_account(&v)?;
let grandpa_key = T::GrandpaAuthorityId::from_account(&v)?;
Some((v, babe_key, grandpa_key))
})
.collect::<Vec<_>>();
Some(validators_with_keys.into_iter().map(|(v, _, _)| v).collect())
}
}
```

2. 슬래싱 메커니즘

BABE 등가 증명 (Equivocation)

```
// BABE 이중 블록 생성 감지
#[derive(Clone, Debug, PartialEq, Eq, Encode, Decode)]
pub struct BabeEquivocationProof<H> {
/// 슬롯 번호
pub slot: Slot,
/// 첫 번째 블록 헤더
pub first_header: H,
/// 두 번째 블록 헤더
pub second_header: H,
```

```
}

impl<T: Config> Pallet<T> {
/// BABE 등가 증명 보고
pub fn report_equivocation(
origin: OriginFor<T>,
equivocation_proof: Box<EquivocationProof<T::Hash, BlockNumberFor<T>>>,
key_owner_proof: T::KeyOwnerProof,
) -> DispatchResultWithPostInfo {
let reporter = ensure_signed(origin)?;
// 등가 증명 검증
Self::verify_equivocation_proof(&equivocation_proof)?;
// 오펜스 보고
let offence = BabeEquivocationOffence {
slot: equivocation_proof.slot,
validator_set_count: key_owner_proof.validator_count(),
offender: equivocation_proof.offender.clone(),
session_index: key_owner_proof.session(),
};
T::EquivocationReportSystem::report_offence(
Some(reporter),
(offence, key_owner_proof),
)?;
```

```
Ok(Pays::No.into())
}
}
```

GRANDPA 등가 증명

```
// GRANDPA 이중 투표 감지
#[derive(Clone, Debug, PartialEq, Eq, Encode, Decode)]
pub struct GrandpaEquivocationProof<H, N> {
/// 세트 ID
pub set_id: SetId,
/// 등가 증명
pub equivocation: Equivocation<H, N, AuthoritySignature, AuthorityId>,
}

// 등가 증명 타입
#[derive(Clone, Debug, PartialEq, Eq, Encode, Decode)]
pub enum Equivocation<H, N, S, Id> {
/// Pre-vote 등가 증명
Prevote(finality_grandpa::Equivocation<Id, finality_grandpa::Prevote<H, N>, S>),
/// Pre-commit 등가 증명
Precommit(finality_grandpa::Equivocation<Id, finality_grandpa::Precommit<H, N>,
}
```

보안 특성 및 공격 저항성

1. BABE 보안 특성

VRF 기반 랜덤성

- 예측 불가능한 슬롯 할당
- 사전 계산 공격 방지
- 검증 가능한 랜덤성

확률적 안전성

- 매개변수 c 로 보안 수준 조절
- 네트워크 지연 허용도 설정
- 적응적 공격자에 대한 저항성

2. GRANDPA 보안 특성

비잔틴 장애 허용

- 1/3 미만의 악의적 노드 허용
- 강한 최종성 보장
- 네트워크 분할 저항성

GHOST 기반 안전성

- 최적의 체인 선택
- 포크 해결 메커니즘
- 장기간 분할 복구

3. 복합 공격 시나리오

```rust
// 복합 공격 대응 메커니즘
impl<T: Config> SecurityAnalysis<T> {
/// Nothing-at-Stake 공격 분석
fn analyze_nothing_at_stake_attack(&self) -> SecurityLevel {
// GRANDPA 최종성이 Nothing-at-Stake 공격 방지
// 최종화된 블록은 되돌릴 수 없음
SecurityLevel::High
}
/// Long Range 공격 분석
fn analyze_long_range_attack(&self) -> SecurityLevel {
// 체크포인트 시스템으로 장거리 공격 방지
// 주기적인 하드 체크포인트 필요
SecurityLevel::Medium
}
/// Eclipse 공격 분석
fn analyze_eclipse_attack(&self) -> SecurityLevel {
// 다양한 피어 연결 필요
// 권한자 발견 메커니즘 중요
SecurityLevel::Medium
}
}
```

성능 최적화 및 튜닝

1. BABE 최적화

```
// 슬롯 시간 최적화
parameter_types! {
// 네트워크 지연을 고려한 슬롯 시간 설정
pub const SlotDuration: u64 = 6000; // 6초
// 에포크 길이 최적화 (보안 vs 유연성)
pub const EpochDuration: u64 = 2400; // 4시간
// 확률 매개변수 (빈 슬롯 확률 = 1-c)
pub const ExpectedBlockTime: u64 = 6000;
}
```

2. GRANDPA 최적화

```
// 투표 타임아웃 최적화
pub struct GrandpaConfig {
/// Pre-vote 타임아웃
pub prevote_timeout: Duration,
/// Pre-commit 타임아웃
pub precommit_timeout: Duration,
/// 라운드 타임아웃
pub round_timeout: Duration,
```

```rust
}


impl Default for GrandpaConfig {
fn default() -> Self {
Self {
prevote_timeout: Duration::from_millis(1000),
precommit_timeout: Duration::from_millis(1000),
round_timeout: Duration::from_millis(2000),
}
}
}
```

3. 네트워크 최적화

```rust
// 메시지 전파 최적화
pub struct NetworkOptimization {
/// 투표 메시지 배치 처리
pub batch_votes: bool,
/// 압축 사용
pub use_compression: bool,
/// 우선순위 큐
pub priority_queue: bool,
```

```
}
```

실제 구현 예제

간단한 합의 설정

```
// 런타임에서 BABE + GRANDPA 설정
impl pallet_babe::Config for Runtime {
type EpochDuration = EpochDuration;
type ExpectedBlockTime = ExpectedBlockTime;
type EpochChangeTrigger = pallet_babe::ExternalTrigger;
type DisabledValidators = Session;
type WeightInfo = ();
type MaxAuthorities = MaxAuthorities;
type KeyOwnerProof = <Historical as KeyOwnerProofSystem<(KeyTypeId, AuthorityId)
type EquivocationReportSystem = pallet_babe::EquivocationReportSystem<
Self,
Offences,
Historical,
ReportLongevity,
>;
}
```

```
impl pallet_grandpa::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type WeightInfo = ();

type MaxAuthorities = MaxAuthorities;

type MaxNominators = MaxNominators;

type MaxSetIdSessionEntries = MaxSetIdSessionEntries;

type KeyOwnerProof = <Historical as KeyOwnerProofSystem<(KeyTypeId, AuthorityId)

type EquivocationReportSystem = pallet_grandpa::EquivocationReportSystem<

Self,

Offences,

Historical,

ReportLongevity,

>;

}
```

다음 단계

이 문서에서 다룬 BABE와 GRANDPA 합의 메커니즘을 바탕으로, 다음 문서에서는 Polkadot 릴레이 체인의 구조와 검증자 시스템을 자세히 살펴보겠습니다:

- 3.1 Polkadot 노드 구조 및 컴포넌트 분석
- 3.2 검증자 시스템 및 스테이킹 메커니즘 분석
- 3.3 온체인 거버넌스 시스템 분석